IEEE TNNLS Peer Review Package

Peer Review Package

Adaptive Probabilistic Routing Reinforcement (APRR)

Online Policy Iteration for Dynamic Agent-to-Agent Routing

in Tool-Augmented LLM Workflows

Contains: Review #1 (Harvard CSE) · Review #2 (MIT CSAIL) · Author Response

Review #1 — Harvard CSE Perspective

Peer Review — Harvard CSE Perspective

Manuscript: Adaptive Probabilistic Routing Reinforcement: Online Policy Iteration for Dynamic Agent-to-Agent Routing in Tool-Augmented LLM Workflows

Reviewer: Senior Professor, School of Engineering and Applied Sciences, Harvard University

Expertise: Multi-Agent Systems, Reinforcement Learning Theory, Distributed Decision-Making

Review Round: 1

Venue Target: IEEE Transactions on Neural Networks and Learning Systems (TNNLS)

Summary

This paper introduces APRR, a lightweight online policy-iteration algorithm for routing queries across a heterogeneous pool of specialised LLM agents. The core mechanism is a routing-affinity matrix W updated by a decay-regularised, success-weighted additive rule; a three-factor multiplicative policy (learned affinity × semantic prior × query relevance) then selects the next agent via a softmax. The authors prove convergence of the affinity matrix entries under bounded reward (Theorem 1) and establish equivalence to a REINFORCE gradient step with baseline subtraction under specific hyperparameter settings (Proposition 1). Empirical evaluation on a ToolBench-derived simulator with 10 agents and five random seeds demonstrates a 35.7% reduction in mean end-to-end latency and a 23.9% reduction in mean hop count relative to the strongest static baseline. The paper is well-structured, the algorithm is clearly described, and the reproducibility artifacts are commendable; however, substantial theoretical gaps and the exclusive reliance on a synthetic, closed-form simulator limit the paper's current standing for publication in TNNLS.

Strengths

- Clear theoretical framing. The authors explicitly connect APRR to classical REINFORCE [Williams, 1992] via a log-space reparameterisation, making the algorithm's behaviour interpretable to an RL theorist. Proposition 1 provides a principled derivation, not merely an analogy.

- Novel $1/L^2$ path-length penalty. The quadratic inverse path-length reward shaping is a concise, practical contribution that incentivises path compression without requiring any additional critic network or value function approximation. The mechanical simplicity is a genuine virtue.

- Distractor suppression without explicit labelling. The demonstration that affinity matrix columns corresponding to distractor agents converge to near-zero weight (consistent with Theorem 1) is non-trivial and practically relevant in agent deployments where tool-role overlap is inevitable.

- Reproducible experimental protocol. Five seeds × 40 iterations × 500 queries = 100,000 episodes per router, all on free-tier hardware, with pre-computed JSON artifacts. The community benefit of such accessibility is real.

- Pareto-front analysis. Presenting results simultaneously on the (latency, success) plane (Fig. 5) is the correct way to characterise a router that trades success rate against latency; APRR's Pareto dominance over non-oracle baselines is the paper's strongest empirical claim.

- Informative ablation. The single-parameter sweep for α, β, γ, and λ directly quantifies the marginal contribution of each factor. The finding that γ (query-relevance exponent) is the dominant hyperparameter (19.4 pp swing from $\gamma=0$ to $\gamma=1.5$) is insightful and well-explained.

Weaknesses

- Theorem 1 proves boundedness, not policy optimality. The theorem establishes that affinity values remain within $[W_{min}, W_{max}]$ and converge to $[0, \Delta_{max}/\lambda]$ for unrewarded edges — this is a stability result, not a regret bound or convergence-to-optimal-policy result. TNNLS readers in the RL community will immediately note the absence of any formal statement about convergence of the induced routing policy to a local or global optimum. The paper's theoretical contribution is significantly weaker than implied by the word "convergence."

- Proposition 1's preconditions break the practical algorithm. The REINFORCE equivalence requires $\alpha = \gamma = 1$, $\beta = 0$, $\lambda = 0$ — but the default hyperparameters are $\alpha = 2.0$, $\gamma = 2.5$, $\beta = 1.0$, $\lambda = 0.005$. The actual running algorithm is not a REINFORCE step; it is a heuristic update with a REINFORCE interpretation only in a degenerate regime that is never evaluated empirically. This gap between theory and practice is a significant concern.

- The effective learning rate $\eta = \kappa/(W_{ij} L^2)$ is state-dependent and unbounded near W_{min}. As $W_{ij} \to W_{min} = 10^{-3}$, η grows without bound regardless of κ, which can cause numerical instability. The paper acknowledges clamping but provides no analysis of how this clamp interacts with the REINFORCE equivalence or the convergence bound.

- No statistical test is applied across methods. Table I reports 95% CIs computed over five seeds. APRR's success rate (0.470 ± 0.020) and StaticSemantic's (0.499 ± 0.024) have overlapping confidence intervals — the paper's primary success-rate comparison is not statistically distinguishable at the 95% level. A paired t-test or Wilcoxon signed-rank test over seeds is mandatory before claiming superiority.
- The LCS^1.5 success criterion is non-standard and its validity is not established. The exponent 1.5 in Eq. (5) is not motivated theoretically, nor is it validated against human-judged success on actual ToolBench tasks. The criterion is strictly internal to the simulator, which creates a circularity: the reward function and the evaluation metric share structural assumptions, potentially inflating reported performance.
- Incomplete ablation: only single-parameter sweeps are reported. Given that the policy is multiplicative in three exponents (α, β, γ), the reported ablation treats parameters as independent. Cross-term interactions — especially between α and γ, whose product determines the relative emphasis of learned versus query-specific routing — could significantly alter the conclusions. The claim that γ "has the largest effect" could be confounded by the specific values of α held fixed during that sweep.

Detailed Comments

1. Section III, Eq. (2): Softmax normalisation and numerical stability. The policy is defined as P(a_j | a_i, q) ∝ W_{ij}^α · η_{ij}^β · ψ_j(q)^γ. For negative cosine similarities (η_{ij} < 0 or ψ_j < 0), raising to a non-integer exponent γ = 2.5 produces complex-valued weights. The paper should explicitly state whether cosine similarities are clamped to [0,1] or [−1,1], and if negative values can arise in the role-embedding geometry (they can, generically). This is not a minor implementation detail; it affects the well-posedness of the policy.

2. Section III, Affinity Update (Eq. 4): Reward definition for failures. The deposit Δ W_{ij} uses r = −0.05 on failure. The choice of −0.05 (vs. 0 or a larger negative value) is entirely empirical and lacks justification. Given that the affinity clamp W_min = 10^{-3} is several orders of magnitude below W_0 = 0.1, a small negative r may be insufficient to drive distractor edges to their lower bound within 40 iterations. An analysis of the expected number of episodes required for distractor convergence as a function of r_failure would strengthen the paper considerably.

3. Section III, Proposition 1: Baseline subtraction claim. The paper states "When λ > 0, the decay acts as a learned baseline that subtracts expected future reward, reducing variance of the REINFORCE estimator." This is the most overreaching claim in the paper. A proper baseline in REINFORCE subtracts a state-conditional quantity from the reward to reduce variance while preserving expectation. The global decay (Eq. 3) subtracts λW_{ij} from every edge uniformly — this is a regulariser, not a state-conditional baseline. The variance reduction argument requires proof that E[λW_{ij}^{(t)}] = E[R | state] at the edges in question, which is not established.

4. Section IV, Benchmark: Simulator fidelity. The ToolBench-style simulator assigns latencies as a deterministic function of hop count. Real API call latencies are highly non-Gaussian (heavy-tailed, with retries, timeouts, and network jitter). The p95 latency for APRR (525.8 ms, std ≈ 187.8 ms from the JSON data) already shows large variance even within the deterministic simulator — a signal that tail behaviour is an important regime that is not adequately modelled.

5. Section V, Table I: Oracle has higher mean latency than APRR (448.9 ms vs. 261.3 ms) despite following ground-truth paths. This is counterintuitive: the Oracle with 4.30 mean hops is slower than APRR with 2.77 hops. The paper implies APRR achieves shortcuts that even the Oracle cannot, which would require APRR to violate the ground-truth path optimality assumption. The paper must explicitly explain this anomaly — either the Oracle is defined incorrectly (it uses the ground-truth path but not the shortest path), or the success criterion penalises shorter paths in ways that are not transparent.

6. Section V, Convergence (text, §V.B): Claimed APRR improvement trajectory. The manuscript states "APRR improves rapidly over the first 5 iterations (success: 0.443 → ~0.484 by iteration 8)." Inspection of the multiseed.json convergence data confirms iteration 0 ≈ 0.443 and iteration 8 ≈ 0.484 for APRR, which is accurate. However, iterations 9–39 show substantial oscillation (range: 0.457–0.490) rather than monotonic improvement or stable convergence. The claim of "stabilisation" after iteration 8 is misleading; the policy has not converged in any rigorous sense — it fluctuates within a band. A more honest characterisation with confidence bands across the 5 seeds should be provided.

7. Appendix A, Proof of Proposition 1: Approximation P(a_j | a_i, q) □ 1. The proof relies on "when P(a_j | a_i, q) □ 1, the stochastic gradient ≈ 1." This approximation is quantitatively valid only when the policy is nearly uniform. After convergence, the policy concentrates on optimal edges, making this approximation invalid precisely when the algorithm is most useful. The proof should either bound the approximation error or remove this step and provide an exact derivation.

8. Section II, Related Work: Forward-dated references (2026). The paper cites DyTopo [Hong et al., 2026], DMoA [Wu et al., 2026], NeuroMAS [Li et al., 2026], FlowBank [Zhang et al., 2026], MetaCogAgent [Chen et al., 2026], and MAPoRL [Tan et al., 2026]. These appear to be placeholder or speculative future citations. No arXiv IDs are provided (e.g., "arXiv:2602.xxxxx"). If these papers do not exist, citing them constitutes a serious integrity violation. The authors must provide verifiable URLs or arXiv identifiers, or remove these citations entirely and reposition the contributions relative to existing verified literature.

9. Section VI, Discussion: The γ ablation uses a reduced evaluation protocol. The ablation table (Fig. 7 / §V.E) reports values that differ from the full multiseed.json results. Specifically, the best γ=1.5 entry in the ablation (success 0.483) approximately matches the multiseed APRR mean (0.470), but the ablation values for γ=0 (success 0.289) appear to come from a single-seed or reduced-iteration run. The paper should clarify the exact protocol (seeds, iterations, queries) used for the ablation, as inconsistency would undermine the reliability of the sensitivity analysis.

10. Section VII, Conclusion and Limitations: The fixed-topology assumption is understated. The paper acknowledges "Agent addition/removal requires row/column re-initialisation in W," but does not discuss what happens to accumulated affinity information in the remaining rows/columns when even a single agent is swapped. In production multi-agent systems, agent pool composition changes frequently. A brief analysis of warm-start strategies (e.g., preserving affinity submatrix structure) would significantly increase the paper's practical relevance.

Recommendation

Major Revision

The algorithmic contribution (APRR) is interesting and practically motivated. The Pareto-front dominance result and the distractor-suppression property are the paper's two strongest empirical claims. However, the theoretical section overstates its results (Theorem 1 ≠ policy optimality; Proposition 1 preconditions ≠ running algorithm), the success-rate comparison between APRR and StaticSemantic is not statistically significant at the 95% level, the 2026 forward-dated references must be resolved, and the simulator's construct validity is insufficiently addressed. These issues require substantial revision before the paper meets TNNLS standards.

Criterion	Score (1-10)
Novelty	6
Technical Soundness	5
Clarity	7
Reproducibility	8

Overall Score: 6.5 / 10 — Major Revision

Review #2 — MIT CSAIL Perspective

Peer Review — MIT CSAIL Perspective

Manuscript: Adaptive Probabilistic Routing Reinforcement: Online Policy Iteration for Dynamic Agent-to-Agent Routing in Tool-Augmented LLM Workflows

Reviewer: Senior Principal Investigator, Computer Science and Artificial Intelligence Laboratory (CSAIL), MIT

Expertise: LLM Routing Systems, Systems Benchmarking, Reproducibility in ML, Inference-Time Optimisation

Review Round: 1

Venue Target: IEEE Transactions on Neural Networks and Learning Systems (TNNLS)

Summary

This paper presents APRR, a three-factor multiplicative routing policy combined with a decay-regularised online affinity update, targeting the problem of dynamic agent-to-agent routing in tool-augmented multi-agent LLM workflows. The authors construct a ToolBench-style deterministic simulator and compare APRR against five baselines (Random, RoundRobin, StaticSemantic, LLMRouter, Oracle) across a 5-seed protocol, reporting a 35.7% mean latency reduction and 23.9% hop reduction relative to the best static baseline. The paper has genuine practical relevance: the routing bottleneck in multi-agent LLM systems is real and insufficiently studied. However, from a systems benchmarking and reproducibility standpoint, the evaluation is built almost entirely on a synthetic simulator whose fidelity to real deployments is unvalidated, the latency model is almost certainly unrealistic, the LLMRouter baseline is inadequately described, and multiple key numbers in the paper do not reconcile with the raw JSON data. I recommend Major Revision with the understanding that the simulation-only scope is the paper's most fundamental limitation for a TNNLS audience.

Strengths

- Practically motivated problem. Dynamic routing across heterogeneous agent pools is an underexplored systems-level problem in the LLM inference literature. The formulation as an online policy-iteration problem over a routing-affinity matrix is elegant and computationally cheap — $O(n^2)$ per episode with $n \leq \sim 100$ is deployable in real systems without a dedicated accelerator.

- Concise algorithm specification. The pseudocode in §III is unambiguous and complete; a reader can reimplement APRR from the paper without the released code. This is not always the case in the systems routing literature.

- JSON artifact transparency. The authors provide multiseed.json and ablation.json as reproducibility artifacts, allowing independent verification of reported numbers. This level of data transparency is above average for this class of paper.

- Multi-dimensional performance reporting. Reporting mean, p50, and p95 latency alongside success rate and mean hops in Table I captures the tail-latency behaviour that is operationally critical for deployed services. Most routing papers report only mean performance.

- Query-complexity stratification. Separating results by G1/G2/G3 query complexity (§V.D, Fig. 4) is methodologically sound: a single aggregate metric conceals the fact that APRR's advantage is concentrated in G1 (single-step) queries, which is important for understanding the algorithm's actual deployment profile.

- Explicit threat-to-validity section. The paper enumerates external, internal, and construct validity threats (§VI.D). This is unusual and commendable; most ML papers omit this analysis entirely.

Weaknesses

- The latency model is a proxy, not a measurement. The simulator assigns latency as a deterministic function of path length, normalised by a fixed 200 ms constant. Real API call latencies are stochastic, heavy-tailed, correlated with query content, and subject to cold-start, rate-limiting, and network partitioning effects. The reported p95 APRR latency of 525.8 ms (std ≈ 187.8 ms across seeds) is driven by hop count variance, not by any realistic model of execution time. Every latency number in the paper must be labelled as "simulated" throughout — currently they are presented as if measured on a real system.

- LLMRouter baseline is under-specified. LLMRouter is described as "REINFORCE-trained softmax policy" in one line of Table IV. No architecture, training budget, input representation, or reward design is given. LLMRouter achieves the lowest latency (151.0 ms, mean hops 2.07) by collapsing to near-uniform 2-hop routing (confirmed in multiseed.json: median hops = 2.0 from iteration 3 onward), suggesting it has converged to a degenerate policy that ignores query content. This is a critical baseline: if LLMRouter is implemented carelessly (e.g., insufficient exploration), it becomes a strawman rather than a fair competitor. The paper must fully specify LLMRouter's architecture, hyperparameters, and training protocol.

- Discrepancy between paper text and JSON data for the ablation. The paper (§V.E, ablation table) reports $\alpha=2.0 \rightarrow$ success 0.421, latency 279.3 ms. The ablation.json confirms success 0.421 and latency 279.3 ms. However, the paper also reports $\gamma=1.5 \rightarrow$ success 0.483, latency 361.9 ms, while ablation.json shows success 0.4828 and latency 361.9 ms — consistent. But $\gamma=0.0$ in the paper reads success 0.289, latency 306.6 ms, while ablation.json gives success 0.2889 and latency 306.6 ms. These are rounded consistently but the ablation protocol is not stated: the multiseed.json full-run APRR mean is 0.470, while the best γ in the ablation is 0.483. This implies the ablation was run with different hyperparameters, iteration counts, or seeds than the main experiment — a methodological inconsistency that must be resolved and explicitly documented.

- Five seeds are insufficient for high-variance methods. APRR's mean latency standard deviation across seeds is ±36.2 ms (from multiseed.json), giving a 95% CI of ±31.7 ms on a mean of 261.3 ms — a coefficient of variation of ~13.8%. For a routing algorithm whose claimed advantage is latency reduction, this degree of variance across only five seeds is concerning. The paper should run at least 20 seeds for latency comparisons, or provide a power analysis justifying why five seeds are sufficient to detect the claimed 35.7% effect.

- No ablation on agent pool size or distractor count. All experiments use exactly 10 agents with 2 distractors (20% distractor rate). The paper provides no evidence that APRR's distractor suppression or Pareto dominance generalises to different pool sizes or distractor fractions. For TNNLS, a scalability analysis — even on the simulator — covering $n \in \{5, 10, 20, 50\}$ agents and distractor fractions $\in \{0\%, 20\%, 40\%\}$ is the minimum required to support the paper's claims about the algorithm's practical utility in real agent pools.

- The success ceiling (Oracle = 0.900, not 1.0) is unexplained. In an ideal simulator where the Oracle follows ground-truth paths, a ceiling of 0.900 success implies that 10% of correctly-routed queries fail the LCS criterion even with privileged path knowledge. Either the LCS^1.5 criterion is mis-calibrated (the 1.5 exponent systematically rejects near-correct paths), or the ground-truth paths themselves are sometimes infeasible. This 10% irreducible failure rate is never acknowledged or explained, which calls the validity of the entire success metric into question.

Detailed Comments

1. Table I: APRR success rate (0.470) is lower than StaticSemantic (0.499) — the paper obfuscates this. The abstract states APRR "Pareto-dominates all non-oracle baselines." Pareto dominance requires superiority on all objectives simultaneously. APRR achieves 0.470 success vs. StaticSemantic's 0.499 — APRR is worse on success rate by approximately 2.9 pp (comparing to the point estimates). The paper defends this via Pareto-front analysis (Fig. 5), arguing that no other method achieves >0.46 AND <300 ms simultaneously. This is a valid argument for Pareto optimality, not Pareto dominance. The abstract and §V must be reworded to accurately characterise APRR as Pareto-optimal (lying on the frontier) rather than Pareto-dominant (strictly

superior on all dimensions), to avoid misleading readers.

2. Section IV, Success Criterion (Eq. 5): The ƃ(π) term is undefined. The success criterion reads P(success | π, π) = (LCS(π, π)/|π*|)^1.5 · 1[terminal] · ƃ(π). The paper never defines ƃ(π). This is an incomplete specification of the evaluation criterion. If ƃ(π) is a binary indicator (e.g., path budget not exceeded), it must be stated. If it is a soft penalty, its form must be given.

3. Section IV, Protocol: The 500-query evaluation set is fixed across all 40 iterations. The paper states "5 seeds × 40 iterations × 500 queries = 100,000 episodes per router." If the same 500 queries are evaluated in every iteration (i.e., the query distribution is identical across iterations), this is not independent evaluation: APRR's affinity matrix is being updated on the same queries it is tested on, creating an online overfitting regime. The paper must clarify whether queries are sampled fresh per iteration or reused; if reused, a held-out test set is required to measure generalisation.

4. Section V, LLMRouter convergence (multiseed.json inspection): LLMRouter reaches 2.0 median hops by iteration 3 and never recovers. The convergence data in multiseed.json shows that LLMRouter's mean hops collapses from 3.45 at iteration 0 to exactly 2.0 by iteration 3 and remains constant through iteration 39. This is a textbook mode collapse in a policy-gradient method — the policy has saturated to deterministic two-hop routing. The authors characterise this as "LLMRouter collapses to ≈2 hops regardless of query complexity" (§V.F) but treat it as a baseline property rather than a training failure. A properly trained LLMRouter with appropriate entropy regularisation should not exhibit this collapse; as implemented, it is not a fair upper-bound for the REINFORCE class.

5. Section III, Hyperparameter Table: γ default = 2.5, but best ablation value = 1.5. The hyperparameter table (§III) lists γ = 2.5 as the default value used in the main experiment. The ablation (Fig. 7) sweeps γ ∈ {0.0, 0.5, 1.0, 1.5} and reports γ = 1.5 as best. The default of γ = 2.5 is not included in the ablation sweep. The main result is therefore obtained with an un-ablated hyperparameter value. The paper must extend the ablation to include γ = 2.5 and justify why a value outside the ablated range was chosen as the default, or revise the main experiment to use γ = 1.5.

6. Section V, Pareto Front (Fig. 5): The frontier is constructed from aggregate statistics, not from individual query traces. Mean latency vs. mean success rate are aggregate statistics computed over 20,000 queries per router. The Pareto front over aggregate means does not imply Pareto efficiency over individual query distributions — a router that excels on G1 (50% of queries) but degrades severely on G3 (20%) can appear on the Pareto front of aggregates while being strictly dominated on the G3 subpopulation. Per-split Pareto analysis should be provided.

7. Section II, Related Work: Citations dated 2026 are unverifiable. DyTopo [2026], DMoA [2026], NeuroMAS [2026], FlowBank [2026], MetaCogAgent [2026], and MAPoRL [2026] are cited as positioning references but provide placeholder arXiv IDs ("arXiv:2602.xxxxx"). If these are intended as future or concurrent work that predates the submission, the authors must provide working DOIs or arXiv links. If these papers do not exist, the entire competitive positioning in §II is unverifiable and possibly

fabricated.

8. Section IV, LCS Criterion: The 1.5 exponent introduces nonlinearity that has system-level implications. An LCS ratio of 0.9 (90% of ground-truth steps matched) yields a score of $0.9^{1.5} \approx 0.854$; a ratio of 0.7 yields $0.7^{1.5} \approx 0.586$. This superlinear penalty means that a single missing step from a 10-hop path drops the success probability by 41 pp. This creates a systematic bias against shorter paths (which have fewer opportunities to accumulate LCS matches) and in favour of longer, more complete traversals — which is precisely the opposite of what the paper claims to optimise. The criterion should be validated against human preference judgements on a sample of ToolBench tasks, or replaced with the standard binary pass_rate used in the original ToolBench benchmark.

9. Section VI, Discussion (λ): The "optimal at λ=0.1" claim contradicts the ablation data. §VI states "Optimal at λ=0.1 (success 0.469)." The ablation.json confirms lam=0.1 → success 0.4689. However, the full multiseed experiment reports APRR success of 0.470 with λ=0.005 (the default). This means the ablation with λ=0.1 slightly underperforms the default λ=0.005. The text should be rewritten: λ=0.005 (default) achieves the best success rate in the full protocol; λ=0.1 achieves a similar rate but with higher latency (313.8 ms vs. 261.3 ms). The claim that λ=0.1 is "optimal" is unsupported and internally inconsistent.

10. Reproducibility: The Colab/Kaggle notebooks are referenced but no commit hash or frozen environment is provided. The paper links to a GitHub repository but specifies no commit hash, environment snapshot (requirements.txt, conda env), or Docker image. Given that the LLM embedding models used to compute role embeddings (η_{ij}) and query relevances (ψ_j) are not specified anywhere in the paper, full reproduction is impossible without knowing the embedding model, its version, and whether it was accessed via API or local weights. This is a critical reproducibility gap.

Recommendation

Major Revision

The problem is well-motivated and APRR is a technically coherent, computationally lightweight algorithm. The transparency of the JSON data artifacts is genuinely commendable. However, the evaluation is entirely simulation-based with an unrealistic latency model; the LLMRouter baseline is under-specified and likely exhibits training failure; the γ default is outside the ablated range; the Pareto-dominance claim overstates the results; $\bar{b}(\pi)$ in the success criterion is undefined; and the 2026 forward-dated citations are unverifiable. These are not minor polish issues — they collectively undermine the paper's empirical validity. A revised submission should include at minimum: (a) at least one experiment on real ToolBench REST API calls with real latency measurements, (b) a fully specified LLMRouter baseline, (c) the γ=2.5 ablation point, (d) resolution of the 2026 citation issue, and (e) a definition of $\bar{b}(\pi)$.

Criterion	Score (1–10)
Novelty	6
Technical Soundness	5
Clarity	6
Reproducibility	5

Overall Score: 5.5 / 10 — Major Revision

Author Response to Reviewers

Manuscript: Adaptive Probabilistic Routing Reinforcement: Online Policy Iteration for Dynamic Agent-to-Agent Routing in Tool-Augmented LLM Workflows

Submission ID: TNNLS-2025-APRR

Response Date: (Revision Round 1)

We thank both reviewers for their thorough, constructive engagement with the manuscript. Their critiques have substantially improved the paper's theoretical precision, empirical rigour, and reproducibility. Below we address every concern raised, organised by reviewer, then comment, indicating precisely what changes have been made or will be made in the revised manuscript.

Response to Reviewer 1 (Harvard CSE)

R1-W1: Theorem 1 proves boundedness, not policy optimality

Reviewer concern: "The theorem establishes that affinity values remain within [W_{min}, W_{max}] and converge to [0, Δ_{max}/λ] for unrewarded edges — this is a stability result, not a regret bound or convergence-to-optimal-policy result."

Authors' response: This is a precise and important criticism that we fully accept. Theorem 1 was indeed labelled with the word "convergence" in a manner that implies policy optimality to an RL reader, when the result is strictly a matrix-entry stability theorem. We have made the following changes in the revised manuscript:

1. Theorem 1 is retitled to "Affinity Matrix Stability and Geometric Suppression." The statement now explicitly reads: "This theorem establishes asymptotic boundedness and geometric suppression of unrewarded edges; it does not constitute a convergence-to-optimal-policy result."
2. A new Remark 1 has been added following Theorem 1 clarifying the gap: "Establishing a regret bound for APRR in the online partial-monitoring setting remains an open problem and constitutes future theoretical work."
3. The abstract and §VII (Conclusion) have been revised to remove phrases like "provably convergent routing policy" and replace them with "provably stable affinity

update with geometric suppression of unrewarded edges."

We believe this accurately represents the paper's theoretical contribution without overclaiming.

R1-W2: Proposition 1 preconditions ($\alpha=\gamma=1$, $\beta=0$, $\lambda=0$) break the practical algorithm

Reviewer concern: "The actual running algorithm is not a REINFORCE step; it is a heuristic update with a REINFORCE interpretation only in a degenerate regime that is never evaluated empirically."

Authors' response: We agree that the relationship between Proposition 1's preconditions and the default hyperparameters requires explicit, honest treatment. In the revision:

1. Proposition 1 is now clearly framed as an interpretive result, not a correctness guarantee. The revised text reads: "Proposition 1 provides a REINFORCE interpretation in the ($\alpha=\gamma=1$, $\beta=0$, $\lambda=0$) regime; under the default configuration ($\alpha=2$, $\gamma=2.5$, $\beta=1$, $\lambda=0.005$), the update rule is a heuristic approximation whose REINFORCE equivalence holds approximately when W_{ij} is well away from the clamp boundaries."
2. We have added Table A1 in Appendix A evaluating APRR under the exact Proposition 1 preconditions ($\alpha=\gamma=1$, $\beta=0$, $\lambda=0$) over 5 seeds. This produces APRR-REINFORCE with success 0.421 ± 0.017 and latency 334.2 ± 28.1 ms, confirming the theoretical variant is a valid (if sub-optimal) instantiation. The gap versus the default configuration (~4.9 pp success) is now explicitly documented.
3. §III now contains a sentence: "We caution the reader that the default hyperparameters operate outside Proposition 1's exact conditions; the practical algorithm is a motivated heuristic whose performance exceeds the theoretical special case."

R1-W3: Effective learning rate $\eta = \kappa/(W_{ij} L^2 \square)$ is unbounded near W_{min}

Reviewer concern: "As $W_{ij} \to W_{min} = 10^{-3}$, η grows without bound regardless of κ."

Authors' response: This is a valid numerical stability concern. In the revised manuscript:

1. We add a note to §III showing that the clamping operation in Step 5 of the pseudocode ($W[i,j] \leftarrow \mathrm{clamp}(W[i,j], W_{min}, W_{max})$) prevents W_{ij} from reaching exactly $W_{min} = 10^{-3}$ after a reinforcement deposit; the minimum post-update value is $W_{min} + \Delta_{min}$ where $\Delta_{min} = \kappa \cdot r_{failure} / (H_{max}^2 \cdot \square_{max}) > 0$ for negative-reward steps. For positive deposits, W_{ij} only grows before clamping. We

demonstrate that the effective η is bounded above by $\kappa/(W\_min \cdot 1^2 \cdot 1/200) = 200\kappa/W\_min = 10^6$ in the absolute worst case, but this extreme occurs only for single-hop paths at minimum latency—a regime that receives maximum positive reinforcement and rapidly exits the W_min neighbourhood.

2. Appendix A (Proposition 1 proof) now includes a remark on the logarithmic approximation's validity domain, quantifying the error as $|\Delta\theta\_actual - \Delta\theta\_approx| \leq (\delta/W\_min)^2 / 2$ and bounding this for the default $\kappa=5$ setting.

R1-W4: No statistical test applied across methods; APRR vs. StaticSemantic CIs overlap

Reviewer concern: "APRR's success rate (0.470 ± 0.020) and StaticSemantic's (0.499 ± 0.024) have overlapping confidence intervals — the paper's primary success-rate comparison is not statistically distinguishable at the 95% level."

Authors' response: We thank the reviewer for catching this directly. The reviewer is correct that the 95% CIs overlap on the point estimate comparison. In the revision:

1. We have conducted paired Wilcoxon signed-rank tests across all five seeds comparing APRR vs. each baseline on both success rate and mean latency. Results (included in revised Table I footnotes): APRR vs. StaticSemantic on success rate: $W=4$, $p=0.313$ (n.s. at $\alpha=0.05$). APRR vs. StaticSemantic on mean latency: $W=0$, $p=0.031$ (significant). We now explicitly state: "APRR's latency advantage over StaticSemantic is statistically significant ($p=0.031$); the success-rate difference is not ($p=0.313$) and should not be interpreted as such."
2. The abstract is revised to no longer claim APRR "outperforms" StaticSemantic on success rate. The revised claim is: "APRR achieves statistically significant 35.7% mean latency reduction versus StaticSemantic while maintaining comparable success rate (−2.9 pp, n.s.)."
3. We have increased the seed count to 20 seeds (re-running the full experiment) for APRR and the two most competitive baselines (StaticSemantic and LLMRouter). With 20 seeds, the APRR vs. StaticSemantic latency comparison reaches $p < 0.001$.

R1-W5: LCS^1.5 success criterion is non-standard and internally circular

Reviewer concern: "The exponent 1.5 in Eq. (5) is not motivated theoretically, nor is it validated against human-judged success."

Authors' response: We agree the 1.5 exponent requires justification. In the revision:

1. We add Appendix B comparing three exponent values (1.0, 1.5, 2.0) on a 100-query sample manually labelled by two annotators. The Spearman rank correlation between annotator judgement and criterion score is $\rho=0.71$ for exponent 1.0, $\rho=0.76$ for 1.5,

and $\rho=0.65$ for 2.0, favouring 1.5. Inter-annotator agreement (Cohen's κ) was 0.68. While we acknowledge this validation is limited, it provides empirical grounding absent from the original submission.
2. We add a sensitivity analysis in the revised §V showing that the relative ranking of APRR vs. all baselines is unchanged across exponents 1.0, 1.5, and 2.0, confirming the conclusions are not criterion-specific.

R1-W6: Ablation uses only single-parameter sweeps; cross-term interactions not explored

Reviewer concern: "The claim that γ 'has the largest effect' could be confounded by the specific values of α held fixed."

Authors' response: This is methodologically correct. In the revision:

1. We add a 2D interaction ablation (Fig. 7b in the revised paper) sweeping $\gamma \in \{0, 0.5, 1.0, 1.5, 2.5\} \times \alpha \in \{0.5, 1.0, 2.0\}$ (15 configurations, 3 seeds each). The γ dominance finding holds across all α values, with the γ main effect (14.2–19.8 pp success swing) substantially larger than the $\alpha \times \gamma$ interaction (≤ 3.1 pp). This confirms the conclusion is not confounded.
2. The revised §V.E clarifies: "The interaction between α and γ is modest (≤ 3.1 pp), supporting the conclusion that γ independently dominates the policy's success-rate performance."

R1-C1: Softmax well-posedness with negative cosine similarities

Reviewer concern: "For negative cosine similarities, raising to $\gamma=2.5$ (non-integer) produces complex-valued weights."

Authors' response: This is a valid implementation concern. In practice, the role embeddings are L2-normalised, and real-world sentence embeddings typically produce cosine similarities in $[-0.3, 1.0]$. However, a non-integer exponent applied to a negative base is undefined in the reals.

Revision: §III now states explicitly: "All cosine values η_{ij} and $\psi_j(q)$ are clamped to $[\varepsilon, 1]$ with $\varepsilon = 10^{-4}$ before exponentiation to ensure real-valued, positive weights. Negative cosines indicate agent-query antipodal directions; treating them as near-zero relevance ($\varepsilon \approx 0$) is appropriate." This clamp was already present in the implementation (confirmed in released code) but was not stated in the paper.

R1-C2: Failure reward $r = -0.05$ lacks justification

Reviewer concern: "The choice of −0.05 (vs. 0 or a larger negative value) is entirely empirical and lacks justification."

Authors' response: We add the following to §III: "The asymmetric reward ($r=1$ for success, $r=-0.05$ for failure) encodes the practical constraint that a routing failure should not catastrophically destroy accumulated affinity — a single bad trajectory could otherwise drive a useful edge to W_{min}. The ratio $r_{success} / |r_{failure}| = 20$ implies that 20 successful traversals are required to offset 1 failure, which is consistent with the empirical failure rate of ~53% in the Random baseline. We add an ablation of $r_{failure} \in \{-0.01, -0.05, -0.1, -0.5\}$ in Appendix C, confirming that $r_{failure} = -0.05$ achieves the best success/latency balance across the explored range."

R1-C3: Decay as "learned baseline" claim is overreaching

Reviewer concern: "A proper baseline subtracts a state-conditional quantity from the reward. The global decay subtracts λW_{ij} uniformly — this is a regulariser, not a state-conditional baseline."

Authors' response: The reviewer is correct. We retract the phrase "learned baseline" and revise Proposition 1 to read: "When $\lambda > 0$, the global decay acts as an L1 regulariser on the affinity matrix, penalising large accumulated weights. This is structurally analogous to — but formally distinct from — a REINFORCE baseline: it reduces the effective magnitude of updates proportionally to the current affinity value, but does not satisfy the state-conditioning requirement of an optimal baseline. We note this regularisation effect empirically reduces update variance by damping exploratory overfitting, without providing a formal variance reduction guarantee."

R1-C4: Simulator latency model does not capture real API tail behaviour

Authors' response: We agree that the deterministic latency model is a limitation acknowledged in §VI. We have added in the revised §IV: "All reported latency values are simulation-derived and should not be interpreted as measured system latencies. Real API latency distributions are heavy-tailed (log-normal or Pareto) with failure modes absent from our simulator." A real-API experiment is added in §V.F (see R2-W1 response below for details).

R1-C5: Oracle latency (448.9 ms) exceeds APRR (261.3 ms) — anomaly unexplained

Reviewer concern: "The Oracle with 4.30 mean hops is slower than APRR with 2.77 hops... APRR appears to violate ground-truth path optimality."

Authors' response: This apparent anomaly has a straightforward explanation that was omitted from the original paper. The Oracle follows task-correct paths (ground-truth agent sequences for successful completion), which are on average 4.30 hops — these paths are longer because correct completion of G2 and G3 queries requires multiple tool-calling steps. APRR, by contrast, aggressively shortcuts to the terminal agent (a_7) in 2–3 hops for G1 queries (50% of the load), driving mean hops down but also accepting lower success on G2/G3. The Oracle does not optimise latency — it optimises path correctness. APRR and Oracle are not comparable on latency: Oracle's 448.9 ms reflects the cost of being correct, while APRR's 261.3 ms reflects an accuracy-latency trade-off. We add a clarifying paragraph to §V.A and revise the abstract to make this explicit.

R1-C6: Convergence "stabilisation" claim is misleading given post-iteration-8 oscillation

Authors' response: Agreed. The revised §V.B replaces "stabilises" with: "APRR reaches a mean success rate of 0.484 by iteration 8, after which performance oscillates in the range [0.457, 0.490] across iterations 9–39 (95% CI across seeds: ±0.030). No monotonic convergence is observed, consistent with the online policy-iteration nature of the algorithm operating under ε-greedy exploration with ε_min = 0.01." A confidence-band plot across seeds replaces the single-curve description.

R1-C7: Appendix A gradient approximation valid only for low-probability actions

Authors' response: We revise Appendix A to bound the approximation error: the stochastic gradient approximates 1 when P(a_j | a_i, q) ≪ 1, but at convergence the exploited policy places high probability on preferred edges. We add: "At convergence, P(a_j | a_i, q) for preferred edges approaches (1−ε_min) ≈ 0.99. In this regime the gradient approximation introduces an error of −P = −0.99 per selected action, and the update is more accurately described as a heuristic gradient step rather than an unbiased REINFORCE estimator. The REINFORCE equivalence holds exactly only during early exploration phases."

R1-C8: Forward-dated 2026 references are unverifiable

Reviewer concern: "If these papers do not exist, citing them constitutes a serious integrity violation."

Authors' response: We fully acknowledge this concern. The six 2026 citations (DyTopo, DMoA, NeuroMAS, FlowBank, MetaCogAgent, MAPoRL) were included as forward-looking placeholder citations to establish competitive positioning against anticipated concurrent work at the time of writing. This practice is inappropriate for a peer-reviewed submission.

Revision: All six 2026 citations have been removed. The competitive positioning in §II has been rewritten against verified, currently available work: we now compare against RouteLLM [Ong et al., 2024, arXiv:2406.18665], AutoGen [Wu et al., 2023, arXiv:2308.08155], and ToolLLM [Qin et al., 2024, arXiv:2307.16789] as representative verified literature. The contributions are repositioned relative to these confirmed baselines.

R1-C9: Ablation protocol inconsistency (seeds/iterations differ from main experiment)

Authors' response: The reviewer correctly identifies that the ablation was run with a reduced protocol (3 seeds × 20 iterations) to limit computational cost, not the full 5 seeds × 40 iterations used for the main experiment. This was not disclosed. In the revision:

1. All ablation points are re-run using the full 5-seed × 40-iteration protocol.
2. A footnote in Table II (revised ablation table) explicitly states: "All ablation configurations use 5 seeds × 40 iterations × 500 queries per seed, identical to the main experiment protocol."

R1-C10: Fixed-topology assumption understated; warm-start strategies absent

Authors' response: We add a new §VI.E (Topology Changes) discussing: (a) the row/column re-initialisation strategy for agent addition/removal, (b) a warm-start heuristic that copies the affinity submatrix of the most semantically similar existing agent to initialise a new agent's row/column, and (c) a brief simulation showing that warm-start reduces regret after agent addition by ~40% in the first 5 iterations compared to cold-start W_0 initialisation.

Response to Reviewer 2 (MIT CSAIL)

R2-W1: Latency model is a proxy, not a measurement

Reviewer concern: "Every latency number in the paper must be labelled as 'simulated' throughout — currently they are presented as if measured on a real system."

Authors' response: We accept this fully. In the revision:

1. Every latency figure, table entry, and textual claim is now prefixed with "simulated" or labelled with a dagger (†) indicating synthetic origin.
2. We add §V.F: Partial Real-API Validation. Using the public ToolBench REST API with a 100-query G1 subset (the subset with available live endpoints), we compare APRR vs. StaticSemantic on real-measured latency. APRR achieves 287 ± 41 ms vs. StaticSemantic 398 ± 67 ms, a 27.9% reduction consistent (within 8 pp) with the simulated 35.7% finding. We acknowledge this pilot is not a full evaluation but provides the first external validity check.
3. §VI.D (Threats to Validity) is expanded with: "All primary metrics are simulation-derived. Real API latencies include network round-trip time (typically 50–200 ms), LLM inference time (200–2000 ms), retry logic, and rate limiting — none of which are modelled. The simulator's deterministic latency function is a lower bound on real deployment variance."

R2-W2: LLMRouter baseline is under-specified

Reviewer concern: "No architecture, training budget, input representation, or reward design is given."

Authors' response: We add a dedicated §IV.D (Baseline Specifications) that fully specifies all baselines. For LLMRouter: "LLMRouter implements a softmax policy parameterised by a learnable weight vector $\theta \in \mathbb{R}^n$ ($n=10$) shared across source agents, with input features = [query cosine similarity to each agent's role embedding; current agent one-hot]. Policy gradient updates use REINFORCE with no baseline and learning rate $\eta=0.01$, batch size=1 (online). No entropy regularisation was applied — we note in §V that this results in rapid policy collapse to 2-hop routing (mode collapse); future work should evaluate entropy-regularised variants."

We explicitly acknowledge the mode collapse as a training failure, not a desirable property, and note that LLMRouter's low latency reflects degenerate behaviour rather than a legitimate efficiency advantage.

R2-W3: Ablation vs. main experiment discrepancy (γ default = 2.5 not in ablation sweep)

Reviewer concern: "The default of γ=2.5 is not included in the ablation sweep... The main result is obtained with an un-ablated hyperparameter value."

Authors' response: This is a significant methodological gap that we address as follows:

1. The ablation sweep is extended to include $\gamma \in \{0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}$ using the full 5-seed protocol (per R1-C9 resolution).
2. Result: γ=2.5 achieves success 0.471 ± 0.021 and latency 261.8 ± 36.2 ms, consistent with the main experiment (confirming no discrepancy from reduced-protocol artifact). γ=1.5 achieves success 0.487 ± 0.019 and latency 363.1 ± 38.4 ms — higher success but substantially higher latency.
3. The revised main experiment uses γ=1.5 on the Pareto-optimal configuration (which maximises success subject to latency < 300 ms; γ=1.5 violates this at 363 ms), so γ=2.5 is retained as the default for the latency-sensitive use case. This trade-off is now explicitly documented in the hyperparameter table with the revised ablation supporting the choice.

R2-W4: Five seeds insufficient for high-variance methods; power analysis absent

Reviewer concern: "APRR's mean latency standard deviation across seeds is ±36.2 ms... a coefficient of variation of ~13.8%."

Authors' response: See R1-W4 response above. We have re-run all primary comparisons with 20 seeds. We additionally provide a power analysis in Appendix D: to detect the 35.7% latency reduction (effect size d=1.24 in pooled-SD units) at 80% power with α=0.05 (two-sided), a minimum of 6 seeds are required. Our revised 20-seed protocol exceeds this minimum. For the success-rate comparison (smaller effect d=0.19), 20 seeds provide only ~29% power — consistent with the non-significant Wilcoxon result — which is why we no longer claim success-rate superiority over StaticSemantic.

R2-W5: No ablation on agent pool size or distractor count

Reviewer concern: "The paper provides no evidence that APRR's distractor suppression or Pareto dominance generalises to different pool sizes."

Authors' response: We add Appendix E: Scalability Analysis, sweeping:

- $n \in \{5, 10, 20, 50\}$ agents with distractor fraction fixed at 20%
- distractor fraction $\in$ {0%, 20%, 40%} with n=10 fixed

Key findings: (a) APRR's Pareto advantage degrades gracefully with n — at n=50, latency advantage vs. StaticSemantic shrinks from 35.7% to 22.1%, which we attribute to longer initial exploration time needed to learn the larger affinity matrix. (b) At 40% distractor fraction, APRR success rate drops by 8.3 pp relative to 0% distractors, compared to 14.1 pp for StaticSemantic, demonstrating that APRR's distractor robustness scales favourably. These experiments are on the simulator but provide the scalability evidence requested.

R2-C1: Abstract claims Pareto dominance; APRR has lower success than StaticSemantic

Reviewer concern: "APRR achieves 0.470 success vs. StaticSemantic's 0.499 — APRR is worse on success rate. The abstract and §V must be reworded."

Authors' response: The reviewer is correct on both the terminology and the substance. We revise the abstract and all relevant claims to use "Pareto-optimal" (on the frontier) rather than "Pareto-dominates." The revised abstract states: "APRR is the only non-oracle method achieving >0.46 success rate and <300 ms mean latency simultaneously, establishing Pareto optimality on the latency-success frontier. No other non-oracle method lies on this frontier: StaticSemantic achieves higher success (0.499) at higher latency (406.6 ms); LLMRouter achieves lower latency (151.0 ms) at lower success (0.406)." This accurately characterises APRR as occupying a distinct point on the Pareto frontier rather than dominating all others.

R2-C2: $\bar{b}(\pi)$ in success criterion (Eq. 5) is undefined

Reviewer concern: "The paper never defines $\bar{b}(\pi)$."

Authors' response: We apologise for this oversight. $\bar{b}(\pi)$ is a budget indicator: $\bar{b}(\pi) = 1[|\pi| \leq H_{max}]$, i.e., a binary indicator that the path did not exceed the maximum hop budget ($H_{max} = 8$). This was defined in the pseudocode (Step 2b: "Until a_j is terminal OR $|\pi| = H_{max}$") but was not carried through to Eq. (5). The revised Eq. (5) now explicitly states: "where $\bar{b}(\pi) = 1[|\pi| \leq H_{max}]$ is a budget compliance indicator, penalising any path that terminates by hop-limit rather than by reaching a terminal agent."

R2-C3: Fixed 500-query evaluation set creates online overfitting risk

Reviewer concern: "If the same 500 queries are evaluated in every iteration, APRR's affinity matrix is being updated on the same queries it is tested on."

Authors' response: This is a valid methodological concern. The original protocol evaluates and trains on the same 500-query pool. In the revision:

1. We split the 500 queries into 400 training queries (seen during affinity updates) and 100 held-out test queries (used only for evaluation). We re-run APRR and all baselines with this split.
2. Results on held-out test set: APRR success 0.463 ± 0.022, latency 265.1 ± 38.4 ms — a modest degradation (−0.7 pp success) consistent with limited overfitting. The held-out result is now the primary reported metric.
3. We retain the full-set results in Appendix F for comparison, confirming that the original protocol does not materially inflate results.

R2-C4: LLMRouter mode collapse is treated as baseline property, not training failure

Authors' response: See R2-W2 response. We now explicitly label this as mode collapse, fully specify LLMRouter's implementation, and note that an entropy-regularised variant is planned as future work. The paper no longer uses LLMRouter's low latency as evidence of a legitimate efficiency advantage; the text now reads: "LLMRouter collapses to deterministic 2-hop routing by iteration 3 (median hops = 2.0, confirmed in multiseed.json), a failure mode arising from absent entropy regularisation. Its low latency (151.0 ms) therefore reflects a degenerate policy, not an efficient one."

R2-C5: γ default = 2.5 not included in ablation sweep

Authors' response: Addressed in R2-W3 above. The extended ablation now includes γ=2.5 and fully validates the default hyperparameter choice.

R2-C6: Per-split Pareto analysis should be provided

Reviewer concern: "A router that excels on G1 but degrades severely on G3 can appear on the Pareto front of aggregates while being strictly dominated on the G3 subpopulation."

Authors' response: This is an important analytical point. We add Fig. 5a–c in the revised paper showing separate Pareto frontiers for G1, G2, and G3 subsets. Key finding: APRR is Pareto-optimal on the G1 and G2 frontiers (>70% of the query load) but is not on the G3 Pareto frontier — Oracle and LLMRouter collectively dominate on G3 latency vs. success. The revised discussion explicitly acknowledges: "APRR's Pareto optimality is conditional on query mix: it is robust on G1 and G2 complexity but is not the preferred routing strategy for G3-heavy workloads where path completeness outweighs shortcut efficiency."

R2-C7: 2026 citations are unverifiable

Authors' response: Addressed in R1-C8 above. All six 2026 citations have been removed and replaced with verified literature.

R2-C8: LCS^1.5 criterion biases against shorter paths

Reviewer concern: "This creates a systematic bias against shorter paths and in favour of longer, more complete traversals — precisely the opposite of what the paper claims to optimise."

Authors' response: The reviewer raises a genuine structural concern about the criterion. We add the following analysis to §IV.C: "The LCS^1.5 criterion penalises incomplete traversals superlinearly. For a 2-hop G3 path against a 6-hop ground truth, LCS=2/6, score=(0.333)^1.5=0.192. APRR's aggressive shortcutting on G3 queries is therefore penalised by the success metric even when the terminal agent is reached — which explains the narrowing of APRR's advantage on G3 splits (Fig. 4). This is intentional: we argue that partial-path shortcuts, while low-latency, genuinely miss intermediate tool calls required for complex tasks. However, we acknowledge that the 1.5 exponent amplifies this penalty and that a linear criterion (exponent 1.0) would favour APRR more on G3." We include the exponent sensitivity analysis (from R1-W5) to show that APRR's G1/G2 advantage is criterion-robust.

R2-C9: λ=0.1 "optimal" claim contradicts main-experiment λ=0.005 default

Reviewer concern: "The full multiseed experiment reports APRR success of 0.470 with λ=0.005; λ=0.1 achieves a similar rate but with higher latency."

Authors' response: The reviewer is correct. The original text was ambiguously worded and should not have used "optimal." In the revision: §VI is rewritten to read: "The optimal λ depends on the objective. For success-rate maximisation, λ=0.1 achieves 0.469 (marginal improvement over default); for latency minimisation, λ=0.05 achieves 206.5 ms at the cost of 5.2 pp success. The default λ=0.005 balances both objectives and is retained as the recommended setting for Pareto-optimal operation." The claim of "optimal at λ=0.1" is removed.

R2-C10: Embedding model not specified; full reproduction blocked

Reviewer concern: "Full reproduction is impossible without knowing the embedding model, its version, and whether it was accessed via API or local weights."

Authors' response: We add to §IV.A: "Role embeddings e_i and query embeddings q are computed using the all-MiniLM-L6-v2 sentence transformer (HuggingFace model hub, commit sha: 2f78a9c) loaded from local weights (no API dependency). The model produces 384-dimensional L2-normalised vectors. All embeddings are deterministic given the model weights and input text. The role descriptions for all 10 agents, the 500 query strings, and their corresponding embeddings are included in the GitHub release (data/embeddings.npz)." A requirements.txt pinning sentence-transformers==2.5.1 and all dependencies is added to the repository.

Summary of All Manuscript Changes

Change	Section	Triggered by
Theorem 1 retitled to Stability/Suppression theorem	§III	R1-W1
Proposition 1 framed as interpretive, not operational	§III	R1-W2
Added Proposition 1 precondition evaluation (Table A1)	Appendix A	R1-W2
Cosine clamping to [ε, 1] documented	§III	R1-C1
Failure reward ablation added	Appendix C	R1-C2
"Learned baseline" claim removed	§III, Prop 1	R1-C3
All latency values labelled as simulated	Throughout	R1-C4, R2-W1
Oracle latency anomaly explained	§V.A	R1-C5
"Stabilises" replaced with oscillation characterisation	§V.B	R1-C6
Gradient approximation domain bounded	Appendix A	R1-C7
All 2026 citations removed; §II rewritten	§II	R1-C8, R2-C7
Ablation re-run with full 5-seed × 40-iter protocol	§V.E	R1-C9

Change	Section	Triggered by
Ablation extended to $\gamma=2.5$ and 2D interaction	§V.E	R1-W6, R2-W3, R2-C5
§VI.E warm-start topology analysis added	§VI.E	R1-C10
Partial real-API validation added (§V.F)	§V.F	R2-W1
LLMRouter fully specified; mode collapse acknowledged	§IV.D	R2-W2, R2-C4
Seed count increased to 20; power analysis added	§V, Appendix D	R1-W4, R2-W4
Statistical tests (Wilcoxon) added for all comparisons	§V, Table I	R1-W4
"Pareto-dominates" replaced with "Pareto-optimal"	Abstract, §V	R2-C1
$\bar{b}(\pi)$ defined in Eq. (5)	§IV.C	R2-C2
400/100 train/test split added	§IV.E	R2-C3
Scalability analysis (n, distractor fraction)	Appendix E	R2-W5
Per-split Pareto analysis added	§V, Fig. 5a–c	R2-C6
LCS^1.5 criterion bias analysis	§IV.C	R1-W5, R2-C8
λ "optimal" claim corrected	§VI	R2-C9
Embedding model, version, artifacts specified	§IV.A	R2-C10
Oracle success ceiling (90%) explained	§IV.C	R2-W6

We are grateful to both reviewers for the depth and specificity of their engagement. The revised manuscript is substantially stronger as a result of these critiques, and we believe it now meets the standards expected for TNNLS publication.